Lab 3 — Nutrients, LU, and Lab 2's algorithm on the machine

Friday, October 30 (~20–40 min with Quiz 8)

Last updated: October 30, 2026.

In class Friday, October 30 with Quiz 8.

Credit: Lab points are for effort. Preview the one-pager; write on the hard copy handed out in class.

Story of the lab. Use the one-liner `A\b` / `solve` → same A, two targets → $A = LU$ (what solvers often save) → **program** Lab 2's algorithm through **RREF** and see the same x → $[A \mid I]$ saves the E_i product as A^{-1} (not what `A\b` prefers) → short power syntax.

Learning objectives

1. Turn a nutrient word problem into $Ax = b$, solve twice with the same A, and check answers.
2. Package elimination as an **LU factorization** (this our first useful matrix factorization).
3. Code Lab 2's algorithm (nonzero pivots only): build each elementary matrix, left-multiply, update M until **RREF**.
4. Run the same process on $[A \mid I]$ and see that the right block is the product of the E_i's.
5. Compute a matrix power A^k and $A^k x_0$ in software.

Math connections

Lab 2 algorithm and elementary matrices; systems (Lectures 21–22); products (Lecture 23). Monday: inverses (Lecture 24).

Key syntax

Task	Octave	Python
Augmented $[A \mid b]$	`M = [A b]`	`M = np.column_stack((A, b))`
Identity	`eye(3)`	`np.eye(3)`
One entry (row i, col j)	`M(i,j)` — **1-based**	`M[i, j]` — **0-based**
Whole column j / last column	`M(:, j)` / `M(:, end)`	`M[:, j]` / `M[:, -1]`
`for` loop	`for k = 1:n ... end`	`for k in range(n):` (body indented)
Skip this pass of a loop	`continue`	`continue`
Print text / a matrix	`disp(...)`	`print(...)`
Number → text (for messages)	`num2str(k)`	`f"... {k} ..."` or `str(k)`
Left-multiply	`M = E * M`	`M = E @ M`
Solve $Ax = b$	`x = A \ b`	`x = np.linalg.solve(A,b)`
Residual	`A*x - b`	`A @ x - b`
Power A^k	`A^k`	`np.linalg.matrix_power(A, k)`

Reading the new bits.

- **Indexing:** Octave `M(2,3)` is row 2, column 3 (counting from 1). Python `M[1, 2]` is the same entry (counting from 0).
- **Colon `:`:** "all rows" or "all columns." So `M(:, end)` = every row, last column (the b / solution column in $[A \mid b]$). In Python, `-1` means last index: `M[:, -1]`.
- **`end` (Octave):** last index in that dimension — here, last column of M.
- **`for`:** repeat a block. Octave wraps the block with `end`. Python uses indentation instead of `end`.
- **`disp` / `num2str`:** `disp` prints; `num2str(k)` turns the number k into text so you can paste it into a message. Python's `print` and f-strings do the same job.

In NumPy, `A**3` is **entrywise** — use `matrix_power` for $A^3 = AAA$.

In Lab 2 we typed `A \ b` / `np.linalg.solve(A,b)` and got an answer. Today we'll code up the algorithm from Lab 2: those commands are doing **elimination**.

Same pantry, two meal targets

You are planning meals from **two foods** only:

- x_1 = number of servings of **oats**
- x_2 = number of servings of **yogurt**

Each serving contributes (toy numbers, not a real diet plan):

	Protein (g)	Fiber (g)
1 serving oats	2	1
1 serving yogurt	3	1

So if you eat x_1 oats and x_2 yogurt, the **total** protein and fiber are

$$\begin{aligned} \text{protein} &= 2x_1 + 3x_2, \\ \text{fiber} &= 1x_1 + 1x_2. \end{aligned}$$

In matrix form that is $Ax = b$, where

$$A = \begin{bmatrix} 2 & 3 \\ 1 & 1 \end{bmatrix}, \qquad x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \qquad b = \begin{bmatrix} \text{protein target} \\ \text{fiber target} \end{bmatrix}.$$

The pantry (matrix A) stays the same every day. What changes is the **target** vector b — how much protein and fiber you want that day.

C3.1 — Monday. You want **26 g protein** and **10 g fiber**.
How many servings of oats and yogurt should you eat?
Write b, solve $Ax = b$ with the one-liner (`A \ b` / `solve`), and check with $Ax - b$ (should be near zero).

Expected: $b = (26, 10)$, $x = (4, 6)$ — 4 oats, 6 yogurt.

C3.2 — Tuesday. Same two foods (same A). Today you want **31 g protein** and **12 g fiber**.
How many servings of each? Solve $Ax' = b'$ and check $Ax' - b'$.

Expected: $b' = (31, 12)$, $x' = (5, 7)$.

Pause. You solved **two** systems that share the same A and differ only in b. Was calling the one-liner twice the smartest use of work? What information about A could a computer **reuse** on Tuesday instead of starting over?

First factorization: $A = LU$

From Lab 2's E_i to two triangular factors

Lab 2 cleared below pivots by left-multiplying elementary matrices E_i. **Stop when A is upper triangular** — call that matrix U. Do **not** scale pivots to 1 or clear above (that is RREF / inverse territory).

For the nutrient matrix, one step does it. Subtract $\frac{1}{2}$ times row 1 from row 2:

$$E = \begin{bmatrix} 1 & 0 \\ -\frac{1}{2} & 1 \end{bmatrix} \quad \Rightarrow \quad EA = U = \begin{bmatrix} 2 & 3 \\ 0 & -\frac{1}{2} \end{bmatrix}.$$

So $EA = U$. Multiply on the left by E^{-1}:

$$A = E^{-1}U.$$

The inverse of "subtract $\frac{1}{2}\cdot$(row 1) from row 2" is "**add** $\frac{1}{2}\cdot$(row 1) to row 2":

$$E^{-1} = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix} = L$$

(lower triangular; the multiplier $\frac{1}{2}$ sits where we cleared). Thus

$$A = LU.$$

Larger n (same idea, more E_i). Clearing below the pivots is a product

$$E_k \cdots E_2 E_1 A = U$$

(upper triangular). Those E_i are the "go down to U" group. Invert and reverse order:

$$A = \underbrace{E_1^{-1} E_2^{-1} \cdots E_k^{-1}}_{L} U.$$

Each E_i^{-1} puts the multiplier **back** (same spot, opposite sign), so L is lower triangular with 1s on the diagonal. A second group of elementary matrices (scale pivots to 1, clear **above**) would finish RREF / $[A \mid I]$ — solvers usually **skip** that group when they only need x.

Using LU to solve $Ax = b$

If $A = LU$, then $Ax = b$ means $LUx = b$. Introduce $y = Ux$:

1. **Forward substitution:** solve $Ly = b$ for y (easy: L lower triangular).
2. **Back substitution:** solve $Ux = y$ for x (easy: U upper triangular).

Same L and U work for every new b (Monday, Tuesday, …). Factor once; each day is two triangular solves.

C3.3.

(a) Form L and U for the nutrient A; check $LU = A$.

(b) For Monday's $b = (26, 10)$, solve $Ly = b$, then $Ux = y$, by hand or in software. Check that x matches C3.1.

Behind `A \ b` (part 1). A typical dense solver does this bookkeeping (often with row swaps: $PA = LU$ if a pivot is 0/tiny). This nutrient A needs no swap. Full RREF every time would redo extra work you do not need for one x.

Goal	Package
Many $Ax = b$ (same foods, new targets)	$A = LU$, then $Ly = b$, $Ux = y$ — close to what `A \ b` reuses
Undo map for every b	$[A \mid I] \to A^{-1}$ (extra E_i group) — useful, but usually *not* how `A \ b` works

Bridge from Lab 2: program the algorithm (no zero pivots)

Lab 2 listed an algorithm while every pivot is nonzero, and walked a 3×3 system by hand to echelon (then sketched full RREF). **Today: put that algorithm on the machine** and push all the way to **RREF** (pivots = 1, zeros above and below).

Reuse Lab 2's system:

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix}, \qquad b = \begin{bmatrix} 6 \\ 11 \\ 14 \end{bmatrix}, \qquad M = [A \mid b] \in \mathbb{R}^{3\times 4}.$$

(You already know the answer will be $x = (1, 2, 3)$.)

Pattern for one step

Pivot at position (k, k) (1-based). To clear entry (i, k):

$$c = \frac{M_{ik}}{M_{kk}}, \qquad E = I_3 \text{ with } -c \text{ in row } i, \text{ column } k, \qquad M \leftarrow EM.$$

To make the pivot 1: scale row k by $1/M_{kk}$ (elementary matrix = I with that factor on the diagonal).

C3.4 — Run RREF (nonzero pivots). Copy–paste **one** of the blocks below, run it, and watch M become $[I \mid x]$. Then compare to the one-liner on the same A, b.

For each pivot column k: scale the pivot to 1 with an elementary matrix, then clear every other entry in that column the same way (Lab 2's E, left-multiply, replace M).

Octave (copy all):

```
A = [1 1 1; 2 3 1; 1 2 3];
b = [6; 11; 14];
M = [A b];          % 3x4 augmented matrix
n = 3;

% for k = 1:n ... end  →  repeat with k = 1, then 2, then 3
for k = 1:n
  % M(k,k) = entry in row k, column k (1-based)
  E = eye(n);
  E(k,k) = 1 / M(k,k);   % put the scale factor on the diagonal of E
  M = E * M;
  % disp prints; num2str(k) turns k into text for the message
  disp(["after scale pivot ", num2str(k)]); disp(M);

  for i = 1:n
    if i == k
      continue;          % skip the pivot row itself
    end
    E = eye(n);
    E(i,k) = -M(i,k);    % -c in row i, column k
    M = E * M;
  end
  disp(["after clear column ", num2str(k)]); disp(M);
end

x = M(:, end)            % all rows (:), last column (end) = solution
x_solver = A \ b
residual = A * x - b
```

Python (copy all):

```
import numpy as np

A = np.array([[1., 1, 1],
              [2, 3, 1],
              [1, 2, 3]])
b = np.array([6., 11, 14])
M = np.column_stack((A, b))  # 3x4 augmented matrix
n = 3

# for k in range(n):  →  k = 0, then 1, then 2  (0-based)
for k in range(n):
```

```
    # M[k, k] = entry in row k, column k (0-based)
    E = np.eye(n)
    E[k, k] = 1.0 / M[k, k]  # put the scale factor on the diagonal of E
    M = E @ M
    print(f"after scale pivot {k+1}\n", M)  # f-string inserts k+1 into the text

    for i in range(n):
        if i == k:
            continue          # skip the pivot row itself
        E = np.eye(n)
        E[i, k] = -M[i, k]    # -c in row i, column k
        M = E @ M
    print(f"after clear column {k+1}\n", M)

x = M[:, -1]                  # all rows (:), last column (-1) = solution
print("x from RREF:", x)
print("x from solve:", np.linalg.solve(A, b))
print("residual:", A @ x - b)
```

Expected: final $M = \left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{array}\right]$, so $x = (1, 2, 3)$ — matching `A \ b` / `solve`, residual ≈ 0.

Behind `A\b` (part 2). Your coded RREF is the Lab 2 algorithm made explicit. Production solvers usually **stop earlier** (LU / $PA = LU$, then triangular solves) instead of forming full RREF every time — same family of row operations, better bookkeeping when only b changes.

Same algorithm, I instead of b

Replace the right-hand column by an identity: form the 3×6 matrix $[A \mid I]$. Run **the same** elementary updates until the left block is I. Call the right 3×3 block B.

C3.5.

(a) Obtain B. Check $B * A$ / `B @ A` equals I (within rounding).

(b) Check $B * b$ equals the x from C3.4 — so $x = Bb$ also works.

(c) B is the product of all the elementary matrices you left-multiplied so we've saved the whole reduction as one matrix.

(d) One sentence: why solvers still prefer `A\b` / LU over "form $B = A^{-1}$, then Bb" for a single (or few) right-hand side(s).

Expected: $BA = I$, $Bb = x$. Monday's name for B is A^{-1}. Forming A^{-1} does extra work when you only need one x.

Powers (syntax practice)

C3.6. Separate from the diet / Lab 2 matrices, enter

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \qquad x_0 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Compute A^2, A^3, and $A^3 x_0$. Spot-check: $A^3 = \begin{bmatrix} 1 & 6 \\ 0 & 1 \end{bmatrix}$, $A^3 x_0 = (6, 1)$.

One-pager

Preview the Lab 3 ticket; write on the hard copy in class.